

Level 2 — Micro-CMS v2

Vulnerability Report

Program: Hacker101 CTF

Target: Level 2 — Micro-CMS v2

Findings: 3 confirmed

Executive Summary

Micro-CMS v2 hardens several behaviors observed in v1: editing requires an authenticated admin session, unauthenticated IDOR-style access to private content was not successful during testing, and page-body rendering encodes HTML special characters (reducing straightforward stored XSS via normal inputs). Despite those improvements, assessment confirmed **three high-impact vulnerabilities**:

1. **SQL injection on the login flow**, enabling authentication bypass and access to private content.
2. **Mass assignment on the page-edit API**, allowing unauthorized control of a visibility/privacy-related field not exposed in the UI.
3. **Blind SQL injection leading to credential disclosure**, recovering a valid admin username and password from the database.

The dominant theme is that authentication and input-handling controls were incomplete: login queries remain injectable; authorization to “be an editor” is not the same as safely binding request fields; and boolean/error differentials in authentication responses enable data exfiltration even when results are not reflected directly.

Detailed offensive walkthrough notes remain in `CTFs/Raw_Notes/`. Proof-of-concept sections here are validation summaries with flag evidence.

Finding 1: Authentication Bypass via SQL Injection on Login

Field	Value
Severity	Critical
Vulnerability Class	SQL Injection → Authentication Bypass
CWE	CWE-89, CWE-287 (Improper Authentication)
Related Flag	Flag 0

Description

Admin authentication accepts username and password over a standard login form and uses those values in a backend SQL query. User-controlled input is not safely parameterized. As a result, an attacker can alter the logical structure of the authentication query.

In this application, injection against the username processing path was sufficient to influence the password value returned (or synthesized) by the database layer such that it matched an attacker-chosen password submitted in the form. The application then treated the login as successful and established an authenticated admin session.

Root cause is string concatenation (or equivalent unsafe composition) of attacker input into SQL used for credential verification. Secondary weaknesses that amplify impact include likely plaintext (or equivalently comparable) password handling in the auth comparison path and insufficient hardening of the login query shape.

Once authenticated, private CMS content that is correctly gated from anonymous users becomes available — demonstrating that access control *after* login cannot compensate for a broken login query.

Proof of Concept

Validation summary (non-actionable):

1. Unauthenticated access to edit functionality and private content was denied (consistent with v2 hardening relative to v1).
2. The login endpoint was assessed for SQL injection. Attacker-controlled input in the username field changed query semantics such that authentication succeeded without knowledge of a legitimate account password.
3. The resulting admin session could access a private page that was previously unavailable.
4. Confirmed with flag evidence from the private page:

```
^FLAG^850f8ccc0c1ef2b62964a0c2e1e167fa167f9ba16b4b4f2b790d4a235a044e43$FLAG$
```

Impact

- **Complete authentication bypass** for the admin interface.
- **Confidentiality**: read private pages and any other admin-only data.
- **Integrity**: edit content, change visibility flags (see Finding 2), or otherwise abuse privileged workflows.
- **Pivot**: a foothold for further injection-based enumeration (see Finding 3).
- In production, this is a Critical business risk: full administrative compromise of the CMS.

Mitigation

1. **Parameterized queries / prepared statements** for all authentication SQL. Bind username (and any lookup keys) as parameters; never concatenate into the query string.
2. **Verify credentials in application code using a slow password hash** (Argon2id / bcrypt / scrypt): fetch the user row by username with a parameter binding, then compare hashes — do not “SELECT a password shaped by user input.”
3. **Use a stable query shape** that cannot be extended via `UNION` /logic manipulation from input.
4. **Generic authentication errors** (“Invalid username or password”) with constant-time-ish handling where practical; avoid revealing whether the username exists *and* avoid returning raw DB errors.
5. **Least-privilege DB account**; disable dangerous DB features the app does not need.
6. **Regression tests and WAF/monitoring** as defense-in-depth — not primary controls. Add an automated test that login rejects SQL metacharacter usernames without granting a session.

Affected Assets

- `POST /login` (username / password form fields)
- Authentication query against the `admins` (or equivalent) table
- Private pages gated behind successful admin login

Finding 2: Mass Assignment on Page Edit (Visibility Control)

Field	Value
Severity	High
Vulnerability Class	Mass Assignment / Excess Field Binding
CWE	CWE-915 (Improperly Controlled Modification of Dynamically-Determined Object Attributes)
Related Flag	Flag 1

Description

After authentication, page editing is performed via an HTTP POST that, in the UI, exposes ordinary fields such as title and body. The backend, however, also recognizes an additional parameter controlling whether a page is public or private (a visibility flag).

That flag is **not presented as an editable control in the normal UI**, but the server accepted and applied it when supplied in the edit request body. This is mass assignment (sometimes discussed alongside parameter pollution): the application binds request parameters onto internal model fields without an explicit allowlist, allowing clients to set sensitive properties that should only change through privileged, deliberate workflows.

The existence of private pages in the product makes this especially dangerous: any principal who can edit a page may also silently alter its confidentiality classification.

Proof of Concept

Validation summary (non-actionable):

1. Authenticated admin session was obtained (via Finding 1 in this assessment path).
2. The edit UI showed title/body fields only; no visible control for public/private state.
3. An edit request that included an additional visibility-related parameter (beyond the UI fields) was accepted by the server and changed page confidentiality state.
4. The challenge treated successful abuse of this control as a confirmed finding:

```
^FLAG^c2e9163fb2e95721081976ed18f99ff0eeaf7ea0da6150b3521e74d8b9779e5c$FLAG$
```

Note: The vulnerable operation is the authenticated edit submission for a page; the visibility field must be processed by that handler to trigger the issue.

Impact

- **Confidentiality downgrade or upgrade abuse:** publish private content, or hide content that should remain public for operational reasons.
- **Policy bypass:** authorization to edit content $\neq$ authorization to change ACL / visibility metadata.
- In production CMS platforms, mass assignment has historically led to privilege escalation (e.g., setting `is_admin=true`) — the same root cause class.

Mitigation

1. **Explicit allowlists for update DTOs.** Only bind `title` and `body` (or other intentionally editable fields). Ignore unknown parameters.
2. **Separate privileged operations.** Changing `public` / `published` / ACL fields should require a dedicated endpoint, role check, and audit log.
3. **Server-side defaults and immutability.** Do not initialize model attributes from the full request map.
4. Framework-specific hardening:
 - Disable global mass assignment;
 - Use strong parameters / update schemas;
 - Mark sensitive columns as non-assignable.
5. **Tests:** submit edit requests containing privileged fields (`public` , `role` , `owner_id` , etc.) and assert they are ignored unless the caller has an explicit entitlement and uses the correct API.

Affected Assets

- Authenticated page edit endpoint (POST to the page edit resource)
- Backend page model fields controlling visibility / privacy
- Private vs public page authorization logic

Finding 3: Credential Disclosure via Blind SQL Injection

Field	Value
Severity	Critical
Vulnerability Class	Blind SQL Injection → Credential Disclosure
CWE	CWE-89, CWE-200
Related Flag	Flag 3

Description

The same login SQL injection surface identified in Finding 1 also supports **data exfiltration** when responses do not directly print query results. Authentication responses expose a usable oracle (for example, differentiated outcomes such as invalid-password style messaging versus other failure modes), which allows inference of database contents through crafted boolean or similar conditions.

Using that oracle, the `admins` table in database `level2` was enumerated. A legitimate administrative account was recovered:

id	username	password
1	shante	juliane

Logging in with those recovered credentials granted access to the associated privileged content and confirmed the finding.

This is distinct from Finding 1's "bypass without knowing a real password": here the injection is used to **steal real credentials**, which remains impactful even if a particular bypass technique is later patched imperfectly, and which is catastrophic when passwords are stored or compared in recoverable form.

Proof of Concept

Validation summary (non-actionable):

1. Login remained injectable (Finding 1).
2. Direct reflective dumping of arbitrary columns via simple multi-column injection attempts was constrained (see Appendix); enumeration therefore relied on blind / oracle-based techniques against authentication responses.
3. The `admins` table was recovered, revealing username `shante` and password `juliane`.

4. Successful login with recovered credentials confirmed authenticity of the dump and yielded:

```
^FLAG^8e2e4015fce89fe87550d669f7802f34e125c7d83dbf099ea84a70bcc06e78cc$FLAG$
```

Impact

- **Full credential compromise** of administrative users.
- **Persistent access** even if one bypass payload is blocked, until passwords are rotated and injection is fixed.
- **Password reuse risk** across systems if operators reuse CMS credentials elsewhere.
- Demonstrates why “we don’t reflect SQL errors” is not sufficient hardening against injection.

Mitigation

1. Apply **all mitigations from Finding 1** (parameterization + hashed password verify). Fixing injection closes both bypass and blind exfiltration.
2. **Hash passwords** with a modern KDF; never store or compare plaintext passwords. Even with injection, hashes (especially slow, salted ones) raise attacker cost — but do not treat hashing as a substitute for stopping injection.
3. **Uniform auth responses** and rate limiting / account lockout to slow oracle abuse.
4. **Monitor authentication anomalies** (high volume of login posts, unusual timing patterns).
5. **Incident response:** on discovery, rotate all CMS credentials, invalidate sessions, and audit admin actions taken during the exposure window.
6. Ensure **page-ID and other query paths** remain parameterized as well (v1 taught that resource IDs can be injectable; v2 login was the live sink).

Affected Assets

- POST /login authentication oracle and SQL query
- Database: level2 / table: admins
- Admin session and private resources reachable after credential use

Appendix: Approaches Tried

This appendix records failed, deferred, and hypothesized work that clarifies what v2 hardened, what remained vulnerable, and how successful findings relate to incomplete defenses. No actionable exploit procedures are included.

Hardening observed relative to Micro-CMS v1

Probe	Result
Unauthenticated IDOR / private page access patterns that worked in v1	Not successful; editing and private content require admin authentication
Reusing body event-handler XSS techniques from v1	Not successful in this build
SQL metacharacter abuse against page URL/identifiers (pattern from v1 Flag 1)	Not successful — page resource queries appear parameterized or otherwise safe in v2

Login injection scoping (supports Findings 1 and 3)

Probe	Result / Learning
Authentication bypass via injectable username influencing returned password material	Confirmed (Finding 1 → Flag 0)
Attempt to force a multi-column injected result (e.g., fabricate both username and password shaped columns)	Internal Server Error — consistent with the backend selecting a single column in the vulnerable query
Mismatch between injected password material and submitted password	“Invalid Password” style response — useful as a boolean/oracle signal for blind techniques
Manual blind enumeration	Deferred initially due to tedium; later completed with assisted blind dumping (Finding 3)

Edit-path analysis (supports Finding 2)

Probe	Result / Learning
UI inspection for hidden fields controlling public/private state	No privileged controls visible in the form

Probe	Result / Learning
Hypothesis that visibility is still a backend field because private pages exist	Confirmed by server accepting an extra visibility-related parameter on edit POST (Finding 2)
Alternate guessed parameter names for publish state	One guessed visibility parameter succeeded; others were unproductive

Hypothesized but incomplete: XSS via database output smuggling (Vulnerability C)

During editing tests it was observed that normal inputs encode `<` to `<` in rendered body output — a meaningful XSS mitigation for ordinary stored input.

A residual hypothesis was documented but **not completed as a confirmed flag finding**: if SQL injection can write raw markup into stored content through a path that skips the same encoding applied to form inputs, a page that encodes only on input (rather than on output) might still execute script when reading from the database.

Status	Notes
Incomplete	Not validated end-to-end in the Raw_Notes; no flag attributed
Defensive relevance	Encode on output (and/or sanitize on read for HTML contexts), not only on input. Input-only encoding fails when data enters the DB through alternate channels (injection, migrations, admin imports, synced services)

Synthesis

v2 successfully closed several v1-era paths (anonymous IDOR, trivial body XSS, page-ID SQLi). Residual Critical risk concentrated in **login SQL injection** (bypass + blind credential theft) and **mass assignment** of confidentiality controls — classic examples of “feature hardening” without fixing unsafe query composition or over-binding of request parameters.